

JAVASCRIPT, IN TABLES

The cycles typed what bites. This is the part you look up — the tables, and every slide the cycles replaced: hoisting, the temporal dead zone, closures and the event loop included. Optional depth, not examined.

► **THE NINE DEMOS ARE YOURS TOO**, in `demos/`. Everything in this deck can be checked against one of them.

SEVEN TYPES, AND ONE OF THEM IS A MISTAKE

PRIMITIVES

`string` · `number` · `boolean` ·
`undefined` · `null` · `symbol` ·
`bigint`

EVERYTHING ELSE

`object` — and arrays, functions
and dates are all objects wearing
hats.

THE MISTAKE

`typeof null === "object"`. A
1995 bug, never fixable. Test with
`value === null`.

```
1 let count = 5;           // number
2 count = 'five';          // still legal. the TYPE lives in the value, not the variable
```

EIGHT VALUES ARE FALSE. EVERYTHING ELSE IS NOT.

sixteen values, sorted by an actual `if`

↺ reload

open ↗

TWO DIFFERENT QUESTIONS ABOUT THE SAME SIXTEEN VALUES

☒ mark the three that arrive from a form or from JSON

`if (value)` – "is it truthy?"

FALSY

false

0

-0

0n

''

null

undefined

NaN

8 of 16

TRUTHY

true

1

-1

'0'

'false'

' '

[]

{}

8 of 16

`value == false` – "is it equal to false?"

EQUAL TO FALSE

false

0

-0

0n

''

'0'

' '

[]

8 of 16

NOT EQUAL TO FALSE

null

undefined

NaN

true

1

-1

'false'

{}

8 of 16

6 values land in opposite bins in the two columns: null · undefined · NaN · '0' · ' ' · []. "Falsy" and "equal to false" are not the same question, and every value above is a value you can actually be holding.

Falsy is not a value and it is not a comparison. It is what the language does when it needs a yes or a no and you handed it something else — in an `if`, in `&&`, in `||`, in a ternary. That is a different question from `== false`, and the two columns above disagree about real values you will actually be holding.

► COVER THE SCREEN AND PREDICT THE NUMBER FIRST. Then look at `[]` — truthy in one column, equal to `false` in the other.

THE EIGHT FALSY VALUES. THERE ARE NO OTHERS.

VALUE	NOTE
<code>false</code>	the obvious one
<code>0, -0, 0n</code>	every kind of zero
<code>""</code>	the empty string. An untouched form field
<code>null</code>	your absence
<code>undefined</code>	the language's absence
<code>NaN</code>	a number that is not one

`"0"`, `"false"`, `[]` and `{}` are all truthy. Three of those four arrive from a form or from JSON.

==, IN THE ORDER THE RULES ARE APPLIED

1 Same type

compare directly. No conversion.

2 null and undefined

equal to each other only — so `value == null` is the one exception worth writing.

3 Object vs primitive

`toString()` it, then start over.

4 Boolean vs anything

the boolean becomes `0` or `1`.

5 String vs number

the **string** becomes a number.

► **HENCE** `'' == 0` **TRUE** AND `'' == '0'` **FALSE**. Not transitive — `demos/01` computes the whole table.

TWO WAYS TO HAVE NOTHING, AND ONE TO HAVE A WRONG NUMBER

undefined

The language's absence. A variable never assigned, a missing property, a function with no `return`.

null

Your absence. You wrote it, on purpose, to mean "known to be empty".

NaN

A `number`. It is not equal to itself. `Number.isNaN(x)` is the test.

WHERE NAN COMES FROM

```
1 Number('12 kg')    // NaN
2 undefined + 1       // NaN
3 [10,9].sort()[0] - 1 // fine, but...
```

NaN never throws. It travels quietly through arithmetic and appears on screen four functions later.

THE TWO NOTHINGS, IN ARITHMETIC

WRITTEN	RESULT	WHY IT MATTERS
<code>null + 1</code>	<code>1</code>	<code>null</code> converts to <code>0</code>
<code>undefined + 1</code>	<code>NaN</code>	<code>undefined</code> converts to <code>NaN</code>
<code>null - 1815</code>	<code>-1815</code>	a book with no year sorts first (cycle 2)
<code>Math.floor(null / 10) * 10</code>	<code>0</code>	an album with no year lands in “decade 0” (the review exemplar)
<code>null === undefined</code>	<code>false</code>	so <code>=== undefined</code> lets a <code>null</code> through
<code>null == undefined</code>	<code>true</code>	the one <code>==</code> worth keeping: <code>value == null</code>

Neither of them throws. Decide what “missing” means — `?? Infinity`, `?? ''`, `== null` — before the arithmetic does it for you.

CONVERTING ON PURPOSE

```
1 Number('42')      // 42
2 Number('')         // 0      ← the empty string is zero. remember this in week 9.
3 Number('42kg')     // NaN
4
5 String(42)         // "42"
6 Boolean('')        // false
7 Boolean('false')   // true    ← a non-empty string. all of them.
8
9 Number.parseInt('42kg', 10) // 42  - stops at the first non-digit
10 Number.parseFloat('3.5rem') // 3.5
```

When a form gives you
"7" and you compare it to 7,
convert — do not let == guess.

CONVERTING ON PURPOSE

WRITTEN	RESULT	WATCH
<code>Number('42')</code>	42	
<code>Number('')</code>	0	the empty field
<code>Number('42kg')</code>	NaN	never throws
<code>parseInt('42kg', 10)</code>	42	always pass the base
<code>String(42)</code>	'42'	
<code>Boolean('false')</code>	true	a non-empty string
<code>Number.isNaN(x)</code>	the test for NaN	<code>x === NaN</code> is always false



TWO OPERATORS THAT REPLACE TEN LINES OF GUARDING

```
1 payload?.results?.[0]?.title      // undefined instead of a TypeError
2 album.rate?.()                    // only if it is actually a function
3
4 const year = album.year ?? 'unknown'; // only null and undefined
5 const year = album.year || 'unknown';  // ALSO 0, '' and NaN ← the bug
6
7 items.find((i) => i.id === id) ?? null // find returns undefined; be explicit
```

A rating of `0` is a rating.

`||` throws it away; `??` keeps it.

? . AND ??

WRITTEN	MEANS
<code>a?.b</code>	<code>undefined</code> if <code>a</code> is null/undefined, instead of throwing
<code>a?.[i]</code>	the same, for an index
<code>a?.()</code>	call it only if it is a function
<code>a ?? b</code>	<code>b</code> only when <code>a</code> is null or undefined
<code>a b</code>	<code>b</code> whenever <code>a</code> is falsy — including <code>0</code> and <code>' '</code>

A rating of `0` is a rating and a price of `0` is free. `||` throws both away.



MODULES: EVERY FILE HAS A DOOR

EXPORTING — A DOOR OUT OF THE FILE

```
1 export const getState = () => state;  
2 export function selectVisible(state) { ... }
```

IMPORTING — AND THE `.js` IS REQUIRED

```
1 import { selectVisible } from './state.js';  
2 // it is a URL, not a module name
```

TWO THINGS THAT BITE

Modules are always **strict**. And `type="module"` is **deferred** — the DOM already exists when it runs, so there is no `DOMContentLoaded` anywhere in your project.

```
1 <script type="module" src="js/app.js"></script>  <!-- and it will not work over file:// -->
```

MODULES

```
1 // state.js
2 export const getState = () => state;
3 export function selectVisible(state) { ... }
4
5 // app.js
6 import { getState, selectVisible } from './state.js';
7 import { getState as read } from './state.js'; // renamed
```

1 The .js is required

it is a URL, not a module name.

2 Relative paths need ./

and modules are strict, so `this` is `undefined` in a plain call.

3 `type="module"` is deferred

the DOM exists; no `DOMContentLoaded`.

4 Not over `file://`

Live Server, always.

YOU KNOW THIS ONE. IT IS NOT THE SAME HERE.

IN C#

A `struct` is copied. A `class` is shared. **You decide, when you write the type.**

IN JAVASCRIPT

The seven primitives are copied. **Everything else is shared.** You do not decide, and there is no keyword.

SO

Every array, every object, every function you pass to anything is the **original**.

There is no `struct`. There is no `ref`. There is one rule, and it is not written anywhere in your code.

SIX METHODS, AND YOU ALREADY KNOW FIVE OF THEM

```
1 items.map((i) => i.title)           // same length, transformed
2 items.filter((i) => i.year > 2000)  // fewer, same items
3 items.find((i) => i.id === wanted)  // one item, or undefined
4 items.some((i) => i.rating === 0)   // a boolean
5 items.every((i) => i.title)         // a boolean
6 items.reduce((sum, i) => sum + i.price, 0) // one value, any shape
```

ALL SIX

Return something new. **None of them touches the original.**

THE SEVENTH

`sort` is not like them — cycle 2 is about exactly that.

ARRAY METHODS — WHAT EACH RETURNS

METHOD	RETURNS	TOUCHES THE ORIGINAL?
<code>map</code>	a new array, same length	no
<code>filter</code>	a new array, fewer items	no
<code>find</code>	one item, or <code>undefined</code>	no
<code>findIndex</code>	an index, or <code>-1</code>	no
<code>some</code> · <code>every</code>	a boolean	no
<code>reduce</code>	one value, any shape	no
<code>slice</code>	a new array	no
<code>sort</code> · <code>reverse</code>	the same array	YES
<code>push</code> · <code>pop</code> · <code>shift</code> · <code>unshift</code> · <code>splice</code>	varies	YES

The two dangerous ones are dangerous *because* they also return the array, so they look like the safe group.

THE ONE THAT IS NOT LIKE THE OTHERS

three ways to sort, and what each one actually compared

↻ reload

open ↗

THREE WAYS TO SORT THE SAME LIST

Data numbers ☐ sort a copy: [...list].sort(...) ☒ show the comparisons

Numbers. Bare sort() and localeCompare both order these as text, so both are wrong here and only the subtraction comparator is right.

list.sort()

STARTED AS [10, 9, 1, 25, 3]

RESULT [1, 10, 25, 3, 9]

ORIGINAL AFTERWARDS [1, 10, 25, 3, 9] ← reordered

WHAT WAS COMPARED

1. "10" vs "9" → first
2. "9" vs "1" → second
3. "1" vs "25" → first
4. "25" vs "3" → first

No comparator. Everything becomes a string first.

list.sort((a, b) => a - b)

STARTED AS [10, 9, 1, 25, 3]

RESULT [1, 3, 9, 10, 25]

ORIGINAL AFTERWARDS [1, 3, 9, 10, 25] ← reordered

WHAT WAS COMPARED

1. 9 vs 10 → first
2. 1 vs 10 → first
3. 1 vs 9 → first
4. 25 vs 9 → second
5. 25 vs 10 → second
6. 3 vs 10 → first
7. 3 vs 9 → first

Numbers only. Subtraction gives the negative / zero / positive that sort wants.

list.sort((a, b) => a.localeCompare(b, 'he'))

STARTED AS [10, 9, 1, 25, 3]

RESULT [1, 10, 25, 3, 9]

ORIGINAL AFTERWARDS [1, 10, 25, 3, 9] ← reordered

WHAT WAS COMPARED

1. 9 vs 10 → second
2. 1 vs 9 → first
3. 1 vs 10 → first
4. 25 vs 10 → second
5. 25 vs 9 → first
6. 3 vs 25 → second
7. 3 vs 9 → first

Text. Knows that 'a' comes before 'B'; and knows the Hebrew alphabet.

sort with no argument converts every element to a string and compares those strings, by code unit. That is not a bug and it has never changed — it is simply not the question you were asking. And it sorts the array you handed it, in place.

► **RUN HEBREW ONLY BEFORE HEBREW + ENGLISH**. The first agrees with everything and teaches the wrong lesson. That is the point.

COMPARATORS

SORTING

WRITE

numbers, ascending

```
(a, b) => a - b
```

numbers, descending

```
(a, b) => b - a
```

text

```
(a, b) => a.localeCompare(b, 'he')
```

a field

```
(a, b) => a.year - b.year
```

missing values last

```
(a.year ?? Infinity) - (b.year ?? Infinity)
```

with a tie-breaker

```
b.rating - a.rating || a.title.localeCompare(b.title, 'he')
```

NEVER

```
list.sort()  
list.sort((a, b) => a.title > b.title)
```

The first compares strings. The second returns a boolean, and `sort` wants a number.

ALWAYS ON A COPY

```
[...list].sort(...)
```

toSorted() : THE COPY, BUILT IN

WHAT THE COURSE TYPES

```
const sorted = [...list].sort(compare);
```

Works in every browser this course meets. A spread, then an in-place sort **of the copy**.

THE SAME, IN ONE CALL

```
const sorted = list.toSorted(compare);
```

Returns a new array; `list` is untouched. Added in ES2023 with `toReversed()`, `toSpliced()` and `with()`; Baseline since July 2023.

Either passes the assignment: the grader holds your input and compares it before and after — it checks behaviour, not spelling.

sort IS STABLE — AND YOU STILL WRITE THE TIE-BREAKER

STABLE, SINCE ES2019

Two items the comparator calls equal (it returns `0`) keep the order they had **before** the sort.

SO WHY A TIE-BREAKER?

Because “the order they had before” is whatever order the data arrived in — the API, the order of adding, the last sort. Ties would jump between renders.

```
|| a.title.localeCompare(b.title)
```

makes the order a decision.

AND WITH NO COMPARATOR

Elements are converted to strings and compared by UTF-16 code units: `[10, 9, 1].sort()` is `[1, 10, 9]`, and `'Emma'` comes before `'dune'`.

TAKING THINGS APART, AND PUTTING THEM TOGETHER

```
1  const { title, year = 'unknown' } = album;    // destructuring, with a default
2  const [first, ...rest] = items;               // and rest
3
4  const next = { ...state, query: 'kid a' };     // spread: copy, then override
5  const grown = [...items, newItem];            // never push into state
6
7  function search(term, { retries = 1, timeoutMs = 8000 } = {}) { ... }
8
9  const label = `${album.title} - ${album.year}`; // template literal
```

ORDER MATTERS

`{ ...state, query }` and `{ query, ...state }` are different. The **last** one wins.

BOTH ARE SHALLOW

A nested object inside the copy is still the **same** object. Act 3, again.

DESTRUCTURING, SPREAD, REST

```
1  const { title, year = 'unknown' } = album;           // with a default
2  const { album: { title: name } } = entry;            // nested, renamed
3  const [first, ...rest] = items;                     // array, with rest
4
5  const next = { ...state, query: 'kid a' };           // copy, then override
6  const grown = [...items, newItem];                  // append without push
7  const merged = { ...a, ...b };                      // b wins on collisions
8
9  function f(a, ...others) { }                        // rest parameter
10 function g({ retries = 1 } = {}) { }                // destructured parameter
```

ORDER DECIDES

`{ ...state, query }` and `{ query, ...state }` are different. The last one wins.

BOTH ARE SHALLOW

A nested object inside the copy is still the **same** object.

var, let, const

	VAR	LET	CONST
Scope	function	block	block
Hoisted	yes, as <code>undefined</code>	yes, uninitialised	yes, uninitialised
Read before declaration	<code>undefined</code> , silently	throws	throws
Reassign the name	yes	yes	no
Mutate the value	yes	yes	yes
Redeclare in the same scope	yes	no	no

let **is** hoisted. It is simply not initialised, and that gap is the temporal dead zone.

FUNCTIONS ARE VALUES. THAT IS NOT A SLOGAN.

```
1 const jobs = [shout, trim, translate]; // stored, like any value
2
3 subscribe(render); // passed, WITHOUT calling it
4
5 function makeComparator(key) { // returned, built to order
6   return (a, b) => a[key].localeCompare(b[key], 'he');
7 }
```

`subscribe(render)` hands over the function. `subscribe(render())` calls it and hands over its **result**.

this, BY CALL SITE

WRITTEN AS

THIS IS

`new Thing()`

the new object (not used in this course)

`fn.call(x) · fn.apply(x) · fn.bind(x)`

`owner.fn()`

`owner` — the thing before the dot

`fn()`

`undefined` in a module

an arrow function

none of the above. Whatever `this` was where it was written

THE BUG

```
button.addEventListener(  
  'click', shelf.render)
```

Passed, not called through the dot. The object is gone.

THE FIX

```
button.addEventListener(  
  'click', () => shelf.render())
```

Keep the dot. Not "use an arrow".

ARROW FUNCTIONS

WRITTEN	RETURNS
<code>(x) => x * 2</code>	<code>x * 2</code>
<code>(x) => { x * 2 }</code>	<code>undefined</code> — braces are a body
<code>(x) => { return x * 2; }</code>	<code>x * 2</code>
<code>(x) => ({ id: x })</code>	an object — note the parentheses
<code>() => ...</code>	no parameters
<code>(a, b) => ...</code>	two parameters

Arrow for a callback. **function** for a method. That is the whole decision.

THREE WAYS TO DECLARE, AND THEY FAIL DIFFERENTLY

the same three lines – and this page throws a real error while it loads

↶ reload

open ↷

THE SAME THREE LINES, THREE DECLARATIONS

WHAT THIS PAGE DID TO ITSELF WHILE LOADING

The browser threw this, uncaught, before you clicked anything. It is in your console too, in red: `Uncaught ReferenceError: Cannot access 'answer' before initialization`

var

let

const

function

typeof, both kinds

```
console.log(x);  
var x = 5;  
console.log(x);
```

WHAT ACTUALLY HAPPENED

line 1 `undefined`

line 3 `5`

No error. `var x` was hoisted to the top of the function and initialised to `undefined`, so line 1 is legal and silent. The mistake is still a mistake — it just gets reported somewhere else, later, usually as NaN.

► **OPEN DEVTOOLS BEFORE THIS ONE.** Run `var` first and notice how safe it looks. Then run `let`.

HOISTED IS NOT THE SAME AS USABLE

`var`

Hoisted to the top of the **function**, initialised to `undefined`. Reading early is legal and silent.

`let` / `const`

Hoisted to the top of the **block**, **not initialised**. Reading early throws. That gap is the temporal dead zone.

function declaration Hoisted with its body. Calling it above its definition works.

THE ONE THAT COSTS MONEY

`var` does not fail. It produces `undefined`, which becomes `NaN`, which reaches the screen. **The error message is a symptom four functions from the cause.**

A CLOSURE IS A FUNCTION THAT REMEMBERS WHERE IT WAS BORN

the puzzle, the fix, and then the version you are graded on

↶ reload

open ↷

THE LOOP EVERYBODY HAS SEEN, AND THE PATTERN YOU ARE ABOUT TO SHIP

1 — THREE FUNCTIONS, ONE VARIABLE

```
for (var i = 0; i < 3; i++) {  
  jobs.push(() => i);  
}  
jobs.map((j) => j());
```

Run

The functions are fine. They all close over the same `i`, and by the time any of them runs, the loop has finished and that one `i` is 3.

2 — THREE FUNCTIONS, THREE VARIABLES

```
for (let i = 0; i < 3; i++) {  
  jobs.push(() => i);  
}  
jobs.map((j) => j());
```

Run

One keyword. `let` creates a fresh binding per iteration, so each function closes over a different one. Nothing about the functions changed.

3 — THE REASON ANYONE CARES

```
function createStore() {  
  let state = { count: 0 };  
  const listeners = new Set();  
  return {  
    getState: () => state,  
    setState(patch) { ... },  
    subscribe(fn) { ... },  
  };  
}
```

setState count + 1

subscribe

unsubscribe

try to reach in

`state` is a plain `let` inside a function that has already returned. Nothing on this page can name it. That is the same closure as panel 1 — and here it is the reason your state cannot be corrupted from outside its file.

► **AFTER PANEL 2, NOTE THE DIFFERENCE:** that was the puzzle version — panel 3 is the one your project is built out of.

THE SAME CLOSURE, IN THE FILE YOU WILL SHIP

```
1 let state = initialState(); // nothing outside this file can name it
2 const listeners = new Set();
3
4 export const getState = () => state;
5 export function setState(patch) {
6   state = { ...state, ...patch };
7   for (const listener of listeners) listener(state);
8 }
9
10 export function subscribe(fn) { listeners.add(fn); return () => listeners.delete(fn); }
```

This is `starter-repo/js/state.js`. You already have it. In week 10 you fill it in.

WORKED EXAMPLE

BUILD THE STORE, IN NINE LINES

IN THE CONSOLE

A counter with private state. `getState`, `setState`, `subscribe`. Then subscribe `console.log` and press it.

THEN BREAK IT ON PURPOSE

Move `let state` outside the function. Everything still works — and now anyone can write to it.

► **NINE LINES, TYPED, NOT PASTED** — at home, in the console. Week 10 builds the real one.

WHERE YOUR CODE RUNS NOW

What it is

One thread, inside a tab, next to a document it can read and change.

Key trait

There is no `main`. Your code runs, finishes, and the page keeps living. Everything after that is a reaction.

Use cases

Week 8: the language only. Week 9: it meets the page.

Tips

The console is your REPL. `console.log`, never `alert`.

THE ONE THAT MATTERS

One thread. While your code runs, nothing else can — not a click, not a timer, not the page repainting. The event-loop slides in this deck are entirely about the consequences.

WHAT DOES THIS PRINT?

```
1 console.log('A');  
2 setTimeout(() => console.log('B'), 0);  
3 Promise.resolve().then(() => console.log('C'));  
4 console.log('D');
```

**Write the four letters
down, then run it to check.**

ONE STACK, TWO QUEUES, ONE RULE

the model steps – then the browser runs it, and the two are compared ↶ reload open ↷

ONE STACK, TWO QUEUES, AND ONE RULE

Program 1 · the classic Step Reset Predict the output Run it for real step 0 of 7

```
console.log('A');

setTimeout(() => {
  console.log('B');
}, 0);

Promise.resolve().then(() => {
  console.log('C');
});

console.log('D');
```

CALL STACK

MICROTASKS

TASKS

THE MODEL PREDICTS

THE BROWSER ACTUALLY DID

Press **Step**, or **Predict the output** to skip to the end.

► **PROGRAM 1, THEN PROGRAM 2.** In program 2 the timer is registered *first* and still runs last. That is the whole mechanism.

THE RULE, IN ONE SENTENCE

- 1 Run**
the synchronous code, to the end. Nothing interrupts it.
- 2 Drain**
every microtask — promises, `await` — including ones added while draining.
- 3 Take one**
task: a timer, a click, a network response.
- 4 Repeat**
from step 2. Microtasks again, before the next task.

**The queues are only consulted
when the stack is empty.
Everything else follows from that.**

`setTimeout(fn, 0)` MEANS "WHEN I AM FREE"

WHAT PEOPLE READ

"Run this in zero milliseconds." It **never means that.**

WHAT IT MEANS

"Put this in the task queue. Run it after the stack empties and the microtasks are done."

MEASURED, IN DEMO 8

A synchronous loop after the timer, and the callback waits for its zero as long as the loop runs — program 4 prints the number on your machine. Your own code decided it.

One thread. If you are busy, nothing else happens — not a click, not a repaint.

THE EVENT LOOP

- 1 Run**
all synchronous code, to the end. Nothing interrupts it.
- 2 Drain microtasks**
`.then`, `await`, `queueMicrotask` — **all of them**, including ones added while draining.
- 3 One task**
a timer, a click, a network response.
- 4 Back to 2**
microtasks again, before the next task.

MICROTASKS

Promises · `await` ·
`queueMicrotask` ·
`MutationObserver`

TASKS

`setTimeout` · `setInterval` ·
events · network callbacks

THE RULE

Neither queue is consulted while anything is on the stack.

FROM WEEK 8, CLAUDE WRITES CODE FOR YOU

WEEKS 1–5 — TUTOR

It explained. You typed every character.

WEEKS 6–7 — ASSISTANT

It produced snippets. You justified every line you kept.

WEEKS 8–12 — PARTNER

It implements. **You decide what, and you check what came back.**

**What is graded is not whether you used it.
It is whether you know what is in your code.**

A SPEC YOU CAN CHECK BEFORE YOU READ ANY CODE

NOT A SPEC

"Filter the list." · "Handle errors properly." · "Make it responsive." · "Show a suitable message."

A SPEC

"An empty query shows everything." · "Matching ignores case." · "With no matches the list is empty and the counter reads 0."

1

Testable

could somebody else tell whether it happened, without asking you?

2

Bounded

one feature. One file. No "and".

3

Edged

empty, missing, duplicated, very long, none.

4

Worded

the actual text that appears on screen, not "a suitable message".

THE FEATURE_SPEC TEST

1 Testable

could somebody else tell whether it happened, without asking you?

2 Bounded

one feature, one file. If the sentence has "and", it is two.

3 Edged

empty · missing · duplicated · very long · none at all.

4 Worded

the text that appears on screen, not "a suitable message".

NOT A SPEC

"Filter the list." · "Handle errors properly." · "Show a suitable message."

A SPEC

"An empty query shows everything." · "With no matches the list is empty and the counter reads 0."

\$7

Filled in **after** the code exists. It is the part that is graded.

TEN DAYS, IN 1995

BRENDAN EICH, NETSCAPE

Ten days to build a scripting language for the browser. It was called **LiveScript**.

THEN MARKETING HAPPENED

Renamed **JavaScript** to ride Java's popularity. The two languages are unrelated.

WHY YOU CARE

The odd parts are 1995 decisions that could never be removed. **The web does not break old pages.**

Every strange rule here is a promise kept to a page written in 1997.

THE FIVE THAT COST MARKS

WRITTEN	WHAT ACTUALLY HAPPENS
<code>if (value == 0)</code>	<code>'</code> and <code>[]</code> pass too
<code>list.sort()</code>	compares strings, and reorders your array
<code>function f(list) { list.sort(...) }</code>	the caller's array is reordered
<code>el.on('click', obj.method)</code>	<code>this</code> is lost
<code>(x) => { x * 2 }</code>	returns <code>undefined</code> , silently

► **EVERY ONE OF THESE HAS A DEMO.** `demos/01`, `07`, `03`, `06`, and cycle 3 of the block.

WHERE TO LOOK, IN ORDER

1 The console

read the message. It names the line and the value.

2 What you typed

`live-code/*/end/` , then `demos/` — pages that measure what they claim.

3 This deck

the tables above.

4 MDN

search "mdn" plus the method name. Not a tutorial site.

5 The primer

`primer/` , if the syntax itself is the problem.

6 In week 13, none of it

which is what the exercises are for.